



Agentic AI Foundation

AGNTCon + MCPCon Europe

Where the **agentic stack** is being built.

Your Agents Need a Router: One Integration for Every Model and Tool

Ignasi Barrera

THE PROBLEM

The second provider is where the glue starts

A single-model agent is a product feature. A multi-provider, tool-using agent becomes a **traffic and policy problem**.

The integration logic grows sideways as each new concern arrives.

Your agent should own **reasoning** — not credentials, retries, token counters, or provider quirks.

`agent.py`

■ OpenAI SDK + provider format

■ Anthropic SDK + credentials

■ Fallback, backoff & health checks

■ MCP discovery, filtering & auth

■ Usage attribution & quota enforcement



AN AGENTIC AI FOUNDATION PROJECT

Agent Router

Models. Tools. Agents.
One router.

Formerly Envoy AI Gateway

Where a router fits in the agent stack

The agent calls one contract. The platform declares traffic intent once. **Envoy** enforces it on the request path.

APPLICATION PLANE

Your agent

OpenAI / Anthropic compatible client
Logical model name
No provider secrets

/v1

/mcp

TRAFFIC CONTROL

Agent Router

Translates provider differences into a declarative, policy-controlled route.

Control plane

Providers · credentials ·
routes · quotas · tools ·
policy

Envoy data plane

Requests · retries · limits ·
telemetry · egress

UPSTREAM ESTATE

Models & tools

Hosted providers
Self-hosted inference
MCP servers

metrics

traces

logs

The application stays boring — on purpose

```
from openai import OpenAI

client = OpenAI(
    base_url="https://router.example/v1",
    api_key=router_token,
)

reply = client.chat.completions.create(
    model="claude-4-sonnet",
    messages=messages,
)

# No provider SDK. No provider key.
# "claude-4-sonnet" is a logical name.
```

The agent speaks a **single OpenAI-compatible API**.

01 Change the base URL once

The SDK integration remains familiar; traffic decisions leave application code.

02 Request a logical model

Routes can override provider-specific model identifiers downstream.

03 Keep secrets behind the router

Provider credentials belong to platform configuration, not agent workloads.

One logical model, many possible paths

Virtualization separates what the agent asks for from what each upstream provider calls the model.

CLIENT REQUEST

claude-4-sonnet

A stable, readable application-level name.
The route maps it to the provider-specific identifier.

Pattern A • split traffic

Use weights to compare providers or distribute compatible capacity.

Bedrock
anthropic.claude-sonnet-4-20250514-v1:0



GCP
claude-sonnet-4@20250514

weight: 50 / 50

Pattern B • protect availability

Use priorities plus retry policy to move to a healthy fallback.

primary
priority: 0



fallback
priority: 1

retry + health signals

weights distribute traffic | priorities express failover order.

Declare the route, not the SDK logic

The logical model name is used for routing; Envoy retries across prioritized backends when the policy says it may.

```
apiVersion: aigateway.envoyproxy.io/v1beta1
kind: AIGatewayRoute
metadata:
  name: claude-resilient
spec:
  rules:
    - matches:
        - headers:
            - name: x-ai-eg-model
              value: claude-4-sonnet
              type: Exact
      backendRefs:
        - name: bedrock
          modelNameOverride: anthropic.claude-sonnet-4-...
          priority: 0
        - name: gcp-anthropic
          modelNameOverride: claude-sonnet-4@20250514
          priority: 1

# BackendTrafficPolicy → generated HTTPRoute
retry: { numAttemptsPerPriority: 1, numRetries: 5 }
retryOn: connect-failure, retrievable-status-codes, 500
```

1 • Virtualize the name

`modelNameOverride` translates the logical request to each provider's exact upstream identifier.

claude-4-sonnet → provider IDs

2 • Express the order

The first backend is priority 0; the next healthy backend is priority 1.

primary → fallback

3 • Make failover actionable

Priority alone is not a retry policy. Attach `BackendTrafficPolicy` to define eligible retry triggers and timing.

connect failure • 5xx • retry

Fine-grained budget control with Quotas and Rate Limits

Configure how much capacity the platform has on each backend and model.
Set how much each team, user or tenant can spend.

Quota Policies

Allow platform owners to set quotas per upstream backends and models, with graceful fallback policies between them when exhausted.

Rate limiting

Fine-grained controls over user token consumption for each available model.
Configure budgets for users and tenants.

EXAMPLE · token budget accounting

Default model bucket · 10,000 tokens / 1h

7,200 used



Tenant "acme" bucket · 5,000 tokens / 1h

4,450 used



next matching requests can be rejected at the policy boundary



Useful controls: per-model limits · header-selected tenant buckets · CEL token-cost expression · shadow mode before enforcement.

Cap the surprise before it happens

Attach a token budget to the AI service backend; carve out tenant-level capacity with a request-header selector.

```
apiVersion: aigateway.envoyproxy.io/v1alpha1
kind: QuotaPolicy
metadata:
  name: team-token-budget
spec:
  targetRefs:
    - group: aigateway.envoyproxy.io
      kind: AIServiceBackend
      name: openai
  perModelQuotas:
    - modelName: gpt-4
      quota:
        costExpression: "input_tokens + output_tokens * 60"
        defaultBucket:
          limit: 10000
          duration: "1h"
    - modelName: gpt-5
      quota:
        costExpression: "input_tokens + output_tokens * 60"
        defaultBucket:
          limit: 1000
          duration: "1h"
```

1 • Match the route exactly

modelName matches the route backend's modelNameOverride

```
gpt-4 ↔ modelNameOverride
```

2 • Price what matters

CEL can weight token classes. Here, output tokens burn six units each.

```
output_tokens * 60
```

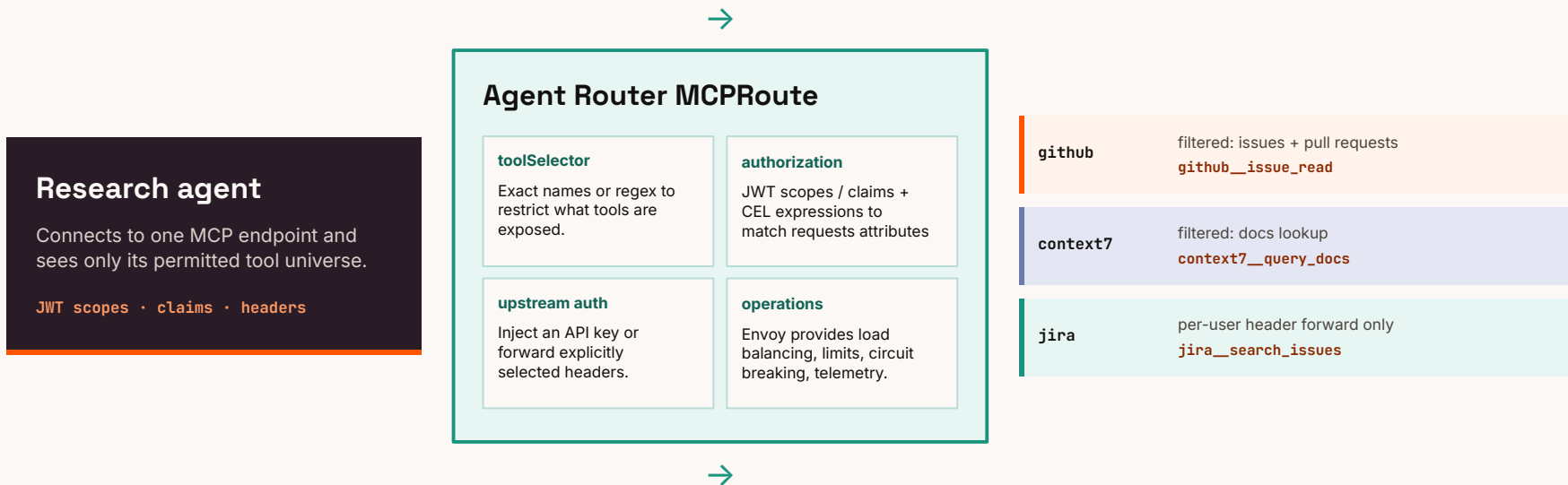
3 • Roll out safely

Configure shadow mode to evaluate and emit telemetry without blocking traffic.

```
observe → tune → enforce
```

Expose the right catalog to the right agent

Aggregate MCP servers behind one endpoint, then filter, discovery and authorize individual tool calls at the gateway.



From laptop to production: same contract, more guardrails

The client API stays the same. The runtime and operational move from dev to prod.

01

Learn locally

Start the standalone router, point an OpenAI-compatible client to its local endpoint, add MCP servers and start experimenting.

```
OPENAI_API_KEY=... aigw run  
localhost:1975/v1
```



02

Prove policy safely

Declare routes, credentials, retries, and policies. Use the rich telemetry data to learn how policies will behave in production..

```
observe → tune → enforce
```



03

Operate the gateway

Deploy Kubernetes resources and the Envoy data plane. Treat provider changes, policy changes, and tracing as platform operations.

```
Gateway API + AI Gateway CRDs
```

Innovate and Experiment

Agent Router is built on Envoy, the industry's most extensible proxy. When a policy doesn't exist yet, you can build it.

**Don't wait for the roadmap.
Extend the data plane to meet your agent's
unique needs.**

dynamic
modules

Dynamic Modules

Write native Go, Rust or C++ extensions for maximum performance when implementing new AI protocols or deep traffic inspection.

ext_proc

External Processing

Implement complex logic that runs out-of-process as an independent service for separate lifecycles and proxy safety.

WASM

WebAssembly

Deploy sandboxed, portable extensions for protocol translation or header manipulation without restarting the proxy.

THE TAKEAWAY

One integration. More choice. Better control.

01

Keep agents focused

The app calls a familiar API and asks for logical models; traffic complexity moves out of its control flow.

02

Make policy declarative

Routing, retry behavior, token budgets, tool exposure, credentials, and telemetry belong in the platform layer.

03

Join the project

Agent Router is an Apache-2.0 Agentic AI Foundation project, powered by Envoy and built in the open.

Docs & quickstart
`theagentrouter.ai`

Source & issues
`github.com/theagentrouter/agent-router`

Community
Discord · Monday community meeting